

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 6	CLOs to be covered: CLO2
Lab Title: Local and Global Variables	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 6

Lab Goals / Objectives:

- Understand the concept of variable scope in Python
- Differentiate between local and global variables
- Learn how to access global variables inside functions
- Understand the global keyword and its usage
- Learn about the globals() and locals() functions
- Understand variable lifetime and memory management
- Apply scope concepts to write better-structured programs

Equipment Required:

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory:

1. What is Variable Scope?

Variable scope defines the region of the program where a variable is accessible. In Python, variables can have either local or global scope depending on where they are declared.

2. Local Variables

A local variable is declared inside a function and is only accessible within that function. It is created when the function is called and destroyed when the function completes execution. Local variables cannot be accessed from outside the function.

```
def my_function():  
    x = 10 # Local variable  
    print(x)  
my_function()  
# print(x) # Error: x is not defined outside the function
```

3. Global Variables

A global variable is declared outside all functions, typically at the top level of the program. It is accessible from any part of the program, including inside functions (for reading). Global variables exist throughout the program's execution.

```
x = 10 # Global variable  
def my_function():  
    print(x) # Can read global variable  
my_function()
```

4. The global Keyword

The global keyword is used inside a function to declare that a variable refers to the global variable, not a local one. This allows functions to modify global variables.

```
count = 0  
def increment():  
    global count
```

```
    count = count + 1
increment()
print(count) # Output: 1
```

5. globals() Function

The `globals()` function returns a dictionary containing all global variables and their values. It can be used to access global variables by name from within a function.

```
x = 40
def show():
    x = 10
    print(x) # Prints local x: 10
    print(globals()["x"]) # Prints global x: 40
```

6. locals() Function

The `locals()` function returns a dictionary containing all local variables and their values in the current scope. It is useful for debugging and introspection.

```
def example():
    a = 1
    b = 2
    print(locals()) # {'a': 1, 'b': 2}
```

7. Variable Shadowing

When a local variable has the same name as a global variable, the local variable 'shadows' the global variable inside the function. Within the function, only the local variable is accessible.

```
x = 20
def func():
    x = 5 # Local x shadows global x
    print(x) # Prints 5
func()
print(x) # Prints 20 (global x unchanged)
```

8. The nonlocal Keyword

The nonlocal keyword is used in nested functions to refer to a variable in the nearest enclosing scope (excluding global). It allows inner functions to modify variables from the outer function.

```
def outer():  
    x = 10  
    def inner():  
        nonlocal x  
        x = 20  
    inner()  
    print(x) # Output: 20
```

9. Best Practices for Variable Scope

- Minimize the use of global variables to avoid unintended side effects
- Use function parameters and return values to pass data between functions
- Use meaningful variable names to avoid naming conflicts
- Document which variables are global when they must be used
- Prefer local variables for temporary calculations

Lab Work:

Task 1: Local vs Global Variable Demonstration

Write a program that demonstrates the difference between local and global variables.

Code Summary:

This task demonstrates how a local variable inside a function shadows the global variable of the same name. The `globals()` function is used to access the global variable even when shadowed.

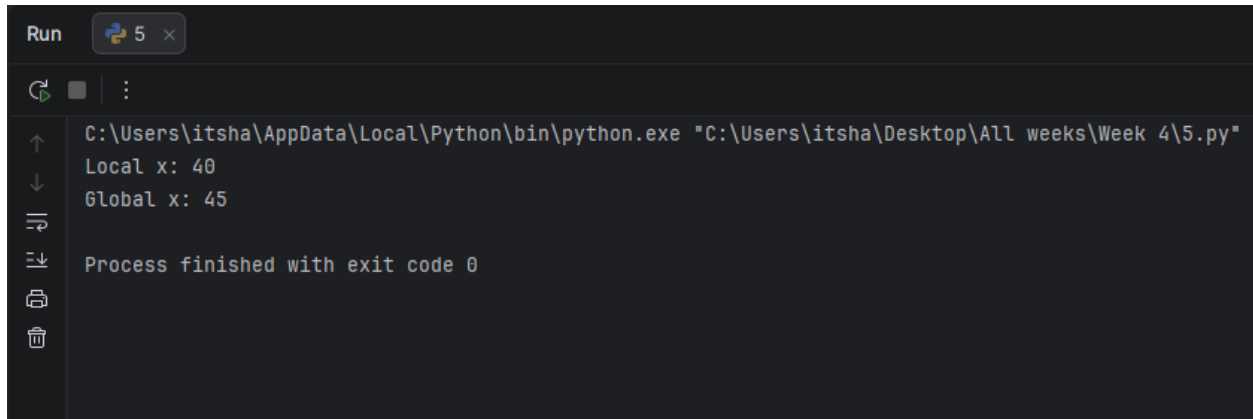
Key Code Lines:

```
# Local vs Global Variable Demonstration  
x = "45"          # Global variable x  
y = "  "          # Global variable y (whitespace)  
  
def welcome():  
    x = 40          # Local variable x (shadows global x)
```

```
print("Local x:", x)          # Prints local: 40
print("Global x:", globals()["x"]) # Prints global: 45

welcome()
```

Output:



```
Run 5 x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"
Local x: 40
Global x: 45
Process finished with exit code 0
```

Task 2: Using the global Keyword

Code Summary:

This task demonstrates how to use the global keyword to modify a global variable from within a function. Without global, the assignment would create a new local variable.

Key Code Lines:

```
# Using global Keyword
counter = 0

def increment():
    global counter
    counter = counter + 1
    print("Counter inside function:", counter)

increment()
increment()
print("Counter outside function:", counter)
```

Output:



```
Run 5 x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"
Counter inside function: 1
Counter inside function: 2
Counter outside function: 2
Process finished with exit code 0
```

Task 3: globals() and locals() Functions

Write a program that demonstrates the use of globals() and locals() built-in functions.

Code Summary:

This task shows how to inspect the global and local namespaces using the globals() and locals() functions, which is useful for debugging and understanding variable scope.

Key Code Lines:

```
# globals() and locals() Demonstration
name = "Hashir"
age = 19

def show_scope():
    city = "Lahore"
    university = "UET"
    print("Local variables:", locals())
    print("Global name:", globals()["name"])
    print("Global age:", globals()["age"])

show_scope()
```

Output:

A screenshot of a Python IDE terminal window. The window has a title bar with 'Run' and a tab labeled '5'. The terminal shows the execution of a Python script. The output is as follows:
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"
Local variables: {'city': 'Lahore', 'university': 'UET'}
Global name: Hashir
Global age: 19
Process finished with exit code 0

Task 4: Nested Functions with nonlocal

Write a program that uses nested functions and the nonlocal keyword.

Code Summary:

This task demonstrates how the nonlocal keyword allows an inner function to modify a variable from its enclosing (non-global) scope.

Key Code Lines:

```
# Nested Functions with nonlocal
def outer_function():
    message = "Hello"

    def inner_function():
        nonlocal message
        message = "Hello, World!"

    print("Before:", message)
    inner_function()
    print("After:", message)

outer_function()
```

Output:

A screenshot of a Python IDE terminal window. The window has a dark background with light-colored text. At the top, there is a 'Run' button and a tab labeled '5'. Below the button, there is a green play icon and a vertical ellipsis. The main area of the terminal shows the command prompt 'C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"' followed by the output 'Before: Hello' and 'After: Hello, World!'. At the bottom, it says 'Process finished with exit code 0'. On the left side of the terminal, there are several icons: a green play icon, a red stop icon, a magnifying glass, a list icon, a document icon, and a trash icon.

Task 5: Practical Scope Exercise

Write a program that calculates the total price including tax using global variables for the tax rate.

Code Summary:

This task demonstrates a practical use case for global variables where the tax rate is defined globally and accessed within a function to perform calculations.

Key Code Lines:

```
# Practical Scope Exercise
TAX_RATE = 0.17 # Global tax rate

def calculate_total(price):
    tax = price * TAX_RATE
    total = price + tax
    return total
```



```
price1 = float(input("Enter price: "))  
print("Total with tax:", calculate_total(price1))
```

Output:



```
Run 5 x  
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"  
Enter price: 1000  
Total with tax: 1170.0  
Process finished with exit code 0
```

Conclusions:

- Successfully understood the concept of variable scope in Python
- Differentiated between local variables (function scope) and global variables (module scope)
- Learned to use the global keyword to modify global variables inside functions
- Understood variable shadowing and how to access global variables using globals()
- Learned about the nonlocal keyword for nested function scopes
- Applied scope concepts to write well-structured and maintainable programs
- Understood best practices for minimizing global variable usage